

Name: Kumaran.M

Reg.No:192525022

Course: Design and Analysis of Algorithm

Course Code: CSA0603

Debugging and Optimization Task

Q21. Debugging Binary Search Code – Boundary Element Failure

1. Problem in the Given Code

The main error is the loop condition:

while low < high:

This condition stops when low == high, so the **last remaining element is never checked**. Therefore, a target at the first or last position can be missed.

The boundary updates themselves are appropriate for the chosen mid, but the loop condition should allow the case where low == high.

2. Corrected Algorithm

1. Set low = 0 and high = len(arr) - 1.
2. Continue searching while low <= high.
3. Calculate the middle index.
4. If arr[mid] == key, return mid.
5. If arr[mid] < key, search the right half using low = mid + 1.
6. Otherwise, search the left half using high = mid - 1.
7. Return -1 if the key is not found.

3. Optimized Python Code

```
def binary_search(arr, key):  
    low = 0  
    high = len(arr) - 1  
  
    while low <= high:  
        mid = low + (high - low) // 2  
  
        if arr[mid] == key:  
            return mid  
  
        elif arr[mid] < key:  
            low = mid + 1  
  
        else:  
            high = mid - 1  
  
    return -1
```

Example

```
arr = [10, 20, 30, 40, 50]
```

```
key = 50
```

```
result = binary_search(arr, key)
```

```
if result != -1:
```

```
    print("Element found at index:", result)
```

```
else:
```

```
    print("Element not found")
```

4. Why the Correction Works

For an array:

[10, 20, 30, 40, 50]

If the key is 50, the search eventually reaches:

low = 4

high = 4

With the original condition $low < high$, the loop stops and index 4 is skipped.

With the corrected condition:

while $low \leq high$:

index 4 is checked, so 50 is correctly found.

5. Complexity Analysis

Case	Complexity	Explanation
Best case	$O(1)$	Key is found at the middle
Average case	$O(\log n)$	Search space is repeatedly halved
Worst case	$O(\log n)$	Key is at a boundary or absent
Space	$O(1)$	Uses only a few variables

6. Edge Cases Handled

- Empty array $\rightarrow$ returns -1
- Single-element array $\rightarrow$ correctly checks the element
- Key at first position $\rightarrow$ correctly found
- Key at last position $\rightarrow$ correctly found
- Key not present $\rightarrow$ returns -1

Conclusion: The primary bug was $low < high$. Replacing it with $low \leq high$ ensures that the final remaining index is examined, making the Binary Search implementation correct and robust.